Custom Prompt Page Configuration Guide

Version: 1.2
Last Updated: January 2026
For: IBM Cognos Analytics Custom Control

Table of Contents

Introduction

The Custom Prompt Page provides a modern, drag-and-drop interface for IBM Cognos Analytics parameter selection. This guide covers the JSON configuration that controls:

- **Left Pane:** Button groups, subgroups, and presets
- **Right Pane:** Card rendering, input types, and validation
- **DataStores:** Query connections for autocomplete values
- **Multi-language:** Localized labels, tooltips, and help text

Key Features

✓ Drag-and-drop parameter selection
✓ Multi-value selection with bubble UI
✓ Date pickers (single, range, from/to)
✓ Required parameter validation
✓ Preset configurations
✓ Multi-language support (German, English, extensible)
✓ Paste support (Excel/CSV comma/newline separated values)

Configuration Overview

Configuration Location

The configuration is passed via the **CustomControl's configuration property** in Cognos:

```
{
  "BaseScriptPath": "/cognos4/samples/javascript/CustomPromptPage/",
  "selectFirstGroupExpanded": true,
  "presets": [...],
  "buttonGroups": [...]
}
```

Configuration Structure

```
Root Configuration
├── BaseScriptPath (string)
├── selectFirstGroupExpanded (boolean)
├── presets (array)
│   └── Preset Objects
└── buttonGroups (array)
    └── Group Objects
        └── groupItems (array)
            ├── Button Objects
            └── Subgroup Objects
                └── buttons (array)
```

Root-Level Properties

BaseScriptPath

Type: `string`
Required: Yes
Default: `/cognos4/samples/javascript/CustomPromptPage/`

Specifies the directory where JavaScript modules are located.

Example:

```
"BaseScriptPath": "/cognos4/samples/javascript/CustomPromptPage/"
```

---

selectFirstGroupExpanded

Type: `boolean`
Required: No
Default: `true`

Controls whether the first button group is expanded by default on page load.

Example:

```
"selectFirstGroupExpanded": true
```

---

presets

Type: `array`
Required: No
Default: `[]`

Array of preset configurations that create multiple parameter cards at once.

See Presets System for details.

---

buttonGroups

Type: `array`
Required: Yes
Default: `[]`

Array of button group objects that appear in the left pane. Each group can contain buttons and subgroups.

See Button Groups Configuration for details.

Button Groups Configuration

Group Object Structure

```
{
  "groupLabel": "Vehicle (VH)",
  "groupLabels": {
    "de": "Fahrzeug (FZ)",
    "en": "Vehicle (VH)"
  },
  "groupIcon": "<svg>...</svg>",
  "defaultExpanded": true,
  "groupItems": [...]
}
```

Group Properties

Property	Type	Required	Description
`groupLabel`	string	Yes	Default group name (fallback)
`groupLabels`	object	No	Localized group names

`groupIcon`	string	No	SVG icon (inline, escaped for JSON)
`defaultExpanded`	boolean	No	Initial expand state (default: `false`)
`tooltip`	string	No	Default tooltip text
`tooltips`	object	No	Localized tooltip text
`groupItems`	array	Yes	Array of buttons and subgroups

Visual Impact

- **Group Header:** Dark background (#1b3e4a), white text, clickable
- **Icon:** Displayed left of label (if provided)
- **Arrow:** ▼ (collapsed) / ▲ (expanded)
- **Tooltip:** Shown on hover over group header

---

groupItems Array

The `groupItems` array contains **buttons** and **subgroups** in the order they should appear.

Identifying Item Types

The system automatically detects item types:

- **Subgroup:** Has `subgroupLabel` property or `buttons` array
- **Button:** Everything else (has `label`, `paramName`, etc.)

Example groupItems

```
"groupItems": [
  {
    "subgroupLabel": "ZP8 Date",
    "buttons": [
      {
        "label": "ZP8 Date (Range)",
        "paramName": "P_ZP8_DateRange",
        "promptType": "dateRange"
      }
    ]
  },
  {
    "label": "Model Year (VH)",
    "paramName": "P_ModelYear",
    "queryName": "qp_ModelYear"
  }
]
```

Subgroup Object Structure

```
{
  "subgroupLabel": "V-Model",
  "subgroupLabels": {
    "de": "V-Modell",
    "en": "V-Model"
  },
  "defaultExpanded": false,
  "tooltip": "Vehicle model codes",
  "tooltips": {
    "de": "Fahrzeugmodell-Codes",
    "en": "Vehicle model codes"
  },
  "buttons": [...]
}
```

Subgroup Properties

Property	Type	Required	Description
`subgroupLabel`	string	Yes	Default subgroup name
`subgroupLabels`	object	No	Localized subgroup names
`defaultExpanded`	boolean	No	Initial expand state (default: `false`)
`tooltip`	string	No	Default tooltip text
`tooltips`	object	No	Localized tooltip text
`buttons`	array	Yes	Array of button objects

Visual Impact

- **Subgroup Header:** Gray background (#d0d4d8), medium weight font
- **Buttons:** Indented 12px from left edge
- **Arrow:** ▼ (collapsed) / ▲ (expanded)
- **Collapsible:** Independent from parent group

Button Properties Reference

Mandatory Properties

Property	Type	Description

`label`	string	Default button text (fallback)
`paramName`	string	Cognos parameter name (must match report parameter)

Exception: Date range (from/to) buttons use `paramNames` object instead of single `paramName`.

Optional Properties

Property	Type	Default	Description
`labels`	object	-	Localized button text
`tooltip`	string	-	Default tooltip text
`tooltips`	object	-	Localized tooltips
`helpText`	string	-	Card-level guidance text
`helpTexts`	object	-	Localized help text
`queryName`	string	-	DataStore name for autocomplete
`useColumn`	integer	0	Column index for USE value
`displayColumn`	integer	1	Column index for DISPLAY value
`promptType`	string	-	Special input type (see below)
`dateFormat`	string	-	Date format (e.g., “yyyy-MM-dd”)
`required`	boolean	false	Mandatory parameter flag
`maxValues`	integer	-	Limit number of selections

Prompt Types Explained

The `promptType` property controls card rendering and input behavior.

Default (undefined)

No `promptType` property

Behavior: - Multi-value bubble input - Autocomplete if `queryName` provided - Enter/Tab to add bubbles - Paste support (comma/newline separated)

Visual:

```
┌──────────────────────────────┐
│ Parameter Name           [×]│
│ Param: P_Example             │
```

```
| ┌──────────────────────────┐ |
| | [Value1 ×] [Value2 ×]    | |
| | Type or select...        | |
| └──────────────────────────┘ |
└──────────────────────────────┘
```

Configuration:

```
{
  "label": "Brand (2-digit) (VH)",
  "paramName": "P_Marke",
  "queryName": "qp_Marke",
  "useColumn": 0,
  "displayColumn": 1
}
```

---

promptType: “text”

Manual text entry without autocomplete

Use Case: Parameters with too many values for DataStore (>10,000 rows)

Behavior: - No DataStore lookup - No datalist autocomplete - Direct text input - Multiple values supported

Visual: Same as default but placeholder says “Type value and press Enter...”

Configuration:

```
{
  "label": "V-Model 6-digit (VH)",
  "paramName": "P_VModel6st",
  "promptType": "text",
  "helpText": "Manual entry - no autocomplete (too many values)"
}
```

---

promptType: “date”

Single date picker

Behavior: - Native HTML5 date input (`<input type="date">`) - Single date selection - Browser’s native date picker UI - Stored as single bubble value

Visual:

```
┌──────────────────────────────┐
| Production Date          [×]|
| Param: P_ProdDate           |
| ┌──────────────────────────┐ |
| | [  ▩  2024-01-15     ] | |
| └──────────────────────────┘ |
└──────────────────────────────┘
```

Configuration:

```
{
  "label": "Production Date",
  "paramName": "P_ProdDate",
  "promptType": "date",
  "dateFormat": "yyyy-MM-dd"
}
```

---

promptType: “dateRange”

Date range with single parameter (uses RangeParameter API)

Use Case: SQL queries with `in_range()` function

Behavior: - Two side-by-side date inputs (FROM/TO) - Single parameter with range structure - Cognos RangeParameter format: `{start: {use: "..."}, end: {use: "..."}}`

Visual:

```
┌──────────────────────────────┐
│ ZP8 Date (Range)          [×]│
│ Param: P_ZP8_DateRange       │
│ FROM          TO             │
│ [2024-01-01] [2024-12-31]  │
└──────────────────────────────┘
```

Configuration:

```
{
  "label": "ZP8 Date (Range)",
  "paramName": "P_ZP8_DateRange",
  "promptType": "dateRange",
  "dateFormat": "yyyy-MM-dd",
  "helpText": "Uses in_range() filter - single parameter with start/end dates"
}
```

Parameter Structure Returned:

```
{
  "parameter": "P_ZP8_DateRange",
  "range": {
    "start": { "use": "2024-01-01" },
    "end": { "use": "2024-12-31" }
  }
}
```

---

promptType: “dateFromTo”

Date range with two separate parameters (uses BETWEEN)

Use Case: SQL queries with `BETWEEN ? AND ?`

Behavior: - Two side-by-side date inputs (FROM/TO) - Two separate parameters - Standard parameter format

Visual: Same as `dateRange`

Configuration:

```
{
  "label": "Recorded on (From/To)",
  "promptType": "dateFromTo",
  "paramNames": {
    "from": "P_Erf.DatFrom",
    "to": "P_Erf.DatTo"
  },
  "dateFormat": "yyyy-MM-dd",
  "helpText": "Select date range when complaints were recorded"
}
```

Parameter Structure Returned:

```
[
  {
    "parameter": "P_Erf.DatFrom",
    "values": [{"use": "2024-01-01", "display": "2024-01-01"}]
  },
  {
    "parameter": "P_Erf.DatTo",
    "values": [{"use": "2024-12-31", "display": "2024-12-31"}]
  }
]
```

Special Properties

required: true

Marks parameter as mandatory

Behavior: - Card auto-rendered on page load - Cannot be removed (no X button) - Shows “★ Required” indicator - Star changes to “✓ Required” (green) when filled

Visual Impact:

```
┌──────────────────────────────┐
│ Brand (2-digit)              │  ← No [×] button
│ Param: P_Marke               │
│ ★ Required                   │  ← Red star when empty
│ ┌──────────────────────────┐ │
│ │                          │ │
│ └──────────────────────────┘ │
└──────────────────────────────┘
```

```
After filling:
| ✓ Required                    |  ← Green checkmark
| ┌──────────────────────────┐ |
| | [MB ×] [BMW ×]           | |
```

Configuration:

```
{
  "label": "Brand (2-digit) (VH)",
  "paramName": "P_Marke",
  "queryName": "qp_Marke",
  "required": true,
  "helpText": "Required: Select at least one vehicle brand"
}
```

---

maxValues: 1

Limits selection to single value

Behavior: - Only 1 bubble allowed - Adding new value clears previous - Enforced in `_createBubble()` method

Visual Impact:

```
Before adding 2nd value:
| [Value1 ×] Type...          |

After adding 2nd value:
| [Value2 ×] Type...          |  ← Value1 automatically removed
```

Configuration:

```
{
  "label": "Primary Brand",
  "paramName": "P_PrimaryBrand",
  "queryName": "qp_Marke",
  "maxValues": 1,
  "helpText": "Select exactly one brand"
}
```

Multi-Language Support

Language Detection

The system auto-detects locale from `oControlHost.locale`: - `"de-DE"` → German (`de`) - `"en-US"` → English (`en`) - Default fallback: English

Localization Pattern

For any user-facing text property:

1. **Singular property:** Default/fallback text
2. **Plural property:** Object with language codes

Example:

```
{
  "label": "Vehicle Model",
  "labels": {
    "de": "Fahrzeugmodell",
    "en": "Vehicle Model"
  },
  "tooltip": "Select model code",
  "tooltips": {
    "de": "Modellcode auswählen",
    "en": "Select model code"
  }
}
```

Localizable Properties

Property	Plural Form	Used In
`label`	`labels`	Buttons, groups, subgroups
`tooltip`	`tooltips`	Buttons, groups, subgroups
`helpText`	`helpTexts`	Card help text
`description`	`descriptions`	Presets
`groupLabel`	`groupLabels`	Button groups
`subgroupLabel`	`subgroupLabels`	Subgroups

Fallback Chain

1. Check `labels[currentLocale]` (e.g., `labels["de"]`)
2. Check `labels["en"]` (English fallback)
3. Check first available key in `labels`
4. Check `label` (singular property)
5. Return empty string

DataStore & Query Configuration

Overview

DataStores provide autocomplete values for parameter selection. They connect to Cognos queries via the `queryName` property.

Critical Configuration Steps

Step 1: Create Query in Cognos

Query Name: Must match `queryName` in JSON
Columns: Minimum 1, recommended 2

Example Query: `qp_Marke`

```
SELECT
  BrandCode,     -- Column 0 (USE value)
  BrandName      -- Column 1 (DISPLAY value)
FROM Brands
ORDER BY BrandName
```

Step 2: Create DataSet in Cognos Report

DataSet Name: Must match `queryName` exactly

1. Open Cognos Report Studio
2. Navigate to **Custom Control** properties
3. Click **“Data Sets”** tab
4. Click **“Add Data Set”**
5. **IMPORTANT:** Name it exactly as `queryName` (e.g., `qp_Marke`)

Step 3: Drag Data Items to DataSet

This is the most common mistake!

✕ **WRONG:** Just selecting the query
✓ **CORRECT:** Dragging data items into the visual DataSet box

How to do it:

1. In DataSet properties, you’ll see a **visual box** representing the DataSet
2. From the query on the left, **drag** `BrandCode` to the box
3. **Drag** `BrandName` to the box
4. Verify both items appear in the DataSet box

Visual Representation:

```
Query: qp_Marke          DataSet: qp_Marke
┌─────────────┐          ┌─────────────┐
│ BrandCode   │─────────>│ BrandCode   │
│ BrandName   │─────────>│ BrandName   │
└─────────────┘          └─────────────┘
       Drag items to DataSet box!
```

If you skip this step: The DataStore will have 0 columns and autocomplete won’t work!

Step 4: Configure JSON Button

```
{
  "label": "Brand (2-digit) (VH)",
  "paramName": "P_Marke",
  "queryName": "qp_Marke",    ← Matches DataSet name
  "useColumn": 0,              ← BrandCode column index
  "displayColumn": 1           ← BrandName column index
}
```

Column Index Configuration

CRITICAL: Column order in the query determines index!

useColumn vs displayColumn

Property	Purpose	Example
`useColumn`	Value sent to Cognos parameter	`"MB"` (code)
`displayColumn`	Value shown to user	`"Mercedes-Benz"` (name)

Default Values: - `useColumn`: `0` (first column) - `displayColumn`: `1` (second column)

Single-Column Queries

If query has only 1 column:

```
{
  "queryName": "qp_Years",
  "useColumn": 0,
  "displayColumn": 0    ← Same as useColumn
}
```

DataStore Validation

Check in browser console:

```
[CustomPromptPage] 📦 DataStore name: qp_Marke
[CustomPromptPage] 📊 Row count: 15
[CustomPromptPage] 📊 Column count: 2
[CustomPromptPage] 📊 Column names: ["BrandCode", "BrandName"]
```

If column count is 0: Data items were not dragged to DataSet!

Query Size Considerations

Small Queries (< 1,000 rows): - Use DataStore with autocomplete - Good performance

Medium Queries (1,000 - 10,000 rows): - Use DataStore (might be slow) - Consider filtering query

Large Queries (> 10,000 rows): - Use `promptType: "text"` (manual entry) - No DataStore needed - User types exact values

Example (large dataset):

```
{
  "label": "V-Model 6-digit (VH)",
  "paramName": "P_VModel6st",
  "promptType": "text",
  "helpText": "Manual entry - no autocomplete (too many values)"
}
```

Presets System

Overview

Presets are “macros” that create multiple parameter cards with a single action.

Preset Object Structure

```
{
  "id": "preset_basic_vehicle",
  "label": "Basic Vehicle Selection",
  "labels": {
    "de": "Basis Fahrzeugauswahl",
    "en": "Basic Vehicle Selection"
  },
  "description": "Load the most common vehicle filter parameters",
  "parameters": [
    { "paramName": "P_ZP8_Jahr" },
    { "paramName": "P_VModel3st" },
    { "paramName": "P_Marke" },
    { "paramName": "P_ISO-LandAus3st" }
  ]
}
```

Preset Properties

Property	Type	Required	Description
`id`	string	Yes	Unique preset identifier
`label`	string	Yes	Default preset name
`labels`	object	No	Localized preset names
`description`	string	No	Default tooltip/description

`descriptions`	object	No	Localized descriptions
`parameters`	array	Yes	Array of parameter objects

Parameters Array

Each parameter object must have:

```
{ "paramName": "P_ExampleParam" }
```

The system will: 1. Find button with matching `paramName` 2. Create empty card for that parameter 3. Disable the source button

---

Visual Impact

Preset Section: - Located at top of left pane - Separated by thick border (#00141f) - Header: "⚡ Presets" - Collapsible (default: expanded)

Preset Buttons: - Same styling as regular buttons (no special color) - Support drag-and-drop + double-click - Do NOT become disabled after use - Tooltip shows description

Interaction: - **Double-click:** Apply preset immediately - **Drag-and-drop:** Drop on right pane to apply - **Duplicate prevention:** Won't create duplicate cards

---

Example Usage

Configuration:

```
{
  "presets": [
    {
      "id": "preset_basic_vehicle",
      "label": "Basic Vehicle Selection",
      "labels": {
        "de": "Basis Fahrzeugauswahl",
        "en": "Basic Vehicle Selection"
      },
      "description": "Load the most common vehicle filter parameters",
      "parameters": [
        { "paramName": "P_ZP8_Jahr" },
        { "paramName": "P_VModel3st" },
        { "paramName": "P_Marke" },
        { "paramName": "P_ISO-LandAus3st" }
      ]
    }
  ]
}
```

Result when clicked: - 4 empty cards created in right pane - 4 corresponding buttons become disabled - User can fill in values as needed

Visual Impact & UI Behavior

Left Pane Rendering

Button Groups

- **Header:** Dark background (#1b3e4a), white text, bold
- **Icon:** SVG displayed inline (if provided)
- **Arrow:** Expand/collapse indicator (▼/▲)
- **Hover:** Clickable cursor

Subgroups

- **Header:** Light gray background (#d0d4d8), semi-bold
- **Indentation:** Buttons indented 12px
- **Arrow:** Independent collapse state

Buttons

- **Default:** Light gray background (#e5e9eb)
- **Hover:** Slight lift effect, shadow
- **Disabled:** 30% opacity, dashed border, no interaction
- **Cursor:** `grab` (draggable) / `grabbing` (during drag)

Presets

- **Section:** Top of pane, separated by border
- **Buttons:** Same as regular buttons (no blue background)
- **Always enabled:** Never become disabled

Right Pane Rendering

Card Structure

```
┌──────────────────────────────┐
│ Parameter Name           [×]│  ← Header with remove button
│ Param: P_ExampleParam       │  ← Parameter info
│ i Help text here...         │  ← Help text (if configured)
│ * Required                  │  ← Required indicator (if applicable)
│ ┌──────────────────────────┐ │
│ │ Input area               │ │  ← Varies by promptType
│ └──────────────────────────┘ │
└──────────────────────────────┘
```

Card Types

Regular Bubble Input:

```
| ┌──────────────────────────┐ |
| | [Val1 ×] [Val2 ×]        | |
| | Type or select...        | |
| └──────────────────────────┘ |
```

Text Input:

```
| ┌──────────────────────────┐ |
| | Type value and Enter... | |
| └──────────────────────────┘ |
```

Date Input:

```
| ┌──────────────────────────┐ |
| | [ ▦ 2024-01-15 ]         | |
| └──────────────────────────┘ |
```

Date Range:

```
| FROM          TO            |
| [2024-01-01] [2024-12-31]   |
```

Interaction Behavior

Drag-and-Drop

1. **Mouse down** on button → Create floating element
2. **Drag** over right pane → Highlight drop zone
3. **Release** over right pane → Create card
4. **Source button** → Becomes disabled

Double-Click

1. **Double-click** button → Immediately create card
2. **Source button** → Becomes disabled

Bubble Creation

1. **Type** value in input
2. **Press Enter** or **Tab** → Create bubble
3. **Select from datalist** → Auto-create bubble
4. **Paste** comma/newline separated → Create multiple bubbles

Bubble Removal

1. **Click X** on bubble → Remove from card
2. **Values array** → Updated immediately
3. **Cognos notified** → `valueChanged()` called

Complete Configuration Examples

Example 1: Simple Multi-Select with Query

```
{
  "label": "Brand (2-digit) (VH)",
  "labels": {
    "de": "Marke (2-stellig) (FZ)",
    "en": "Brand (2-digit) (VH)"
  },
  "tooltip": "Select vehicle brand(s)",
  "tooltips": {
    "de": "Fahrzeugmarke(n) auswählen",
    "en": "Select vehicle brand(s)"
  },
  "paramName": "P_Marke",
  "queryName": "qp_Marke",
  "useColumn": 0,
  "displayColumn": 1,
  "helpText": "Select from dropdown or type to filter",
  "helpTexts": {
    "de": "Aus Dropdown wählen oder tippen zum Filtern",
    "en": "Select from dropdown or type to filter"
  }
}
```

Creates: - Button with German/English label - Card with autocomplete from `qp_Marke` DataStore - Multi-value selection with bubbles - Help text displayed in card

---

Example 2: Required Single-Value Parameter

```
{
  "label": "Production Year",
  "labels": {
    "de": "Produktionsjahr",
    "en": "Production Year"
  },
  "paramName": "P_Year",
  "queryName": "qp_Years",
  "required": true,
  "maxValues": 1,
  "helpText": "Required: Select exactly one year",
  "helpTexts": {
    "de": "Erforderlich: Genau ein Jahr auswählen",
    "en": "Required: Select exactly one year"
  }
}
```

Creates: - Auto-rendered card (required) - No remove button - Single value only (maxValues: 1) - Red star → green checkmark when filled

---

Example 3: Date Range with in_range()

```
{
  "label": "ZP8 Date (Range)",
  "labels": {
    "de": "ZP8-Datum (Bereich)",
    "en": "ZP8 Date (Range)"
  },
  "tooltip": "Select ZP8 date range for production filtering",
  "tooltips": {
    "de": "ZP8-Datumsbereich für Produktionsfilterung auswählen",
    "en": "Select ZP8 date range for production filtering"
  },
  "paramName": "P_ZP8_DateRange",
  "promptType": "dateRange",
  "dateFormat": "yyyy-MM-dd",
  "helpText": "Uses in_range() filter - single parameter with start/end dates",
  "helpTexts": {
    "de": "Verwendet in_range() Filter - einzelner Parameter mit Start-/Enddatum",
    "en": "Uses in_range() filter - single parameter with start/end dates"
  }
}
```

Creates: - Card with FROM/TO date pickers - Single parameter with RangeParameter structure - Localized labels for date fields

---

Example 4: Date Range with BETWEEN

```
{
  "label": "Recorded on (From/To)",
  "labels": {
    "de": "Erfasst am (Von/Bis)",
    "en": "Recorded on (From/To)"
  },
  "promptType": "dateFromTo",
  "paramNames": {
    "from": "P_RecordedDateFrom",
    "to": "P_RecordedDateTo"
  },
  "dateFormat": "yyyy-MM-dd",
  "helpText": "Select date range when records were created",
  "helpTexts": {
    "de": "Datumsbereich auswählen, wann Datensätze erstellt wurden",
    "en": "Select date range when records were created"
  }
}
```

Creates: - Card with FROM/TO date pickers - Two separate parameters - Standard parameter values format

---

Example 5: Text Input (Large Dataset)

```
{
  "label": "V-Model 6-digit (VH)",
  "labels": {
    "de": "V-Modell 6-stellig (FZ)",
    "en": "V-Model 6-digit (VH)"
  },
  "tooltip": "Vehicle model 6-digit code - manual entry",
  "tooltips": {
    "de": "Fahrzeugmodell 6-stelliger Code - manuelle Eingabe",
    "en": "Vehicle model 6-digit code - manual entry"
  },
  "paramName": "P_VModel6st",
  "promptType": "text",
  "helpText": "Manual entry - no autocomplete (too many values)",
  "helpTexts": {
    "de": "Manuelle Eingabe - keine Autovervollständigung (zu viele Werte)",
    "en": "Manual entry - no autocomplete (too many values)"
  }
}
```

Creates: - Card with plain text input - No DataStore/autocomplete - Multi-value support with bubbles

---

Example 6: Complete Button Group

```
{
  "groupLabel": "Vehicle (VH)",
  "groupLabels": {
    "de": "Fahrzeug (FZ)",
    "en": "Vehicle (VH)"
  },
  "groupIcon": "<svg width=\"24\" height=\"24\">...</svg>",
  "defaultExpanded": true,
  "groupItems": [
    {
      "subgroupLabel": "ZP8 Date",
      "subgroupLabels": {
        "de": "ZP8 Datum",
        "en": "ZP8 Date"
      },
      "defaultExpanded": false,
      "buttons": [
        {
          "label": "ZP8 Date (Year)",
          "labels": {
            "de": "ZP8-Datum (Jahr)",
            "en": "ZP8 Date (Year)"
          },
          "paramName": "P_ZP8_Jahr",
          "queryName": "qp_ZP8_Jahr",
          "helpText": "Multi-select: Choose one or more production years"
        }
      ]
    },
    {
```

```
      "label": "Model Year (VH)",
      "labels": {
        "de": "Modelljahr (FZ)",
        "en": "Model Year (VH)"
      },
      "paramName": "P_ModelYear",
      "queryName": "qp_ModelYear",
      "helpText": "Select vehicle model year(s)"
    }
  ]
}
```

Creates: - Expandable group with icon - Subgroup with 1 button (collapsed by default) - 1 direct button under group - All localized for German/English

---

Troubleshooting Guide

Problem: Autocomplete Not Working

Symptoms: - Typing in input shows no suggestions - No dropdown appears

Causes & Solutions:

1. **DataStore not created**
   - ✓ Create DataSet in Cognos with matching `queryName`
2. **Data items not dragged to DataSet**
   - ✓ Drag data items from query into visual DataSet box
   - Check console: `Column count: 0` indicates this problem
3. **Wrong queryName**
   - ✓ Ensure `queryName` in JSON matches DataSet name exactly
4. **Query returns 0 rows**
   - ✓ Check query SQL in Cognos
   - Check console: `Row count: 0`

Problem: Button Not Appearing

Symptoms: - Button configured but not visible in left pane

Causes & Solutions:

1. **Missing mandatory properties**
   - ✓ Ensure `label` and `paramName` exist
2. **Wrong groupItems structure**
   - ✓ Check button is in `groupItems` array
   - ✓ Not inside `buttons` array at group level
3. **Subgroup not converted**

- ✓ Verify subgroup has `subgroupLabel` or `buttons` array

Problem: Wrong Use/Display Values

Symptoms: - Display value sent to Cognos instead of use value - Parameter filter not working correctly

Causes & Solutions:

1. **Swapped column indices**
   - ✓ Check `useColumn` vs `displayColumn`
   - ✓ Verify query column order
2. **Single-column query**
   - ✓ Set both `useColumn` and `displayColumn` to `0`

Verification:

```
Console output:
Row 0: display="Mercedes-Benz", use="MB"
                      ↑ displayColumn    ↑ useColumn
```

Problem: Date Parameter Not Working

Symptoms: - Date selected but parameter not passed - Error in console about parameter format

Causes & Solutions:

1. **Wrong promptType**
   - ✓ Use `dateRange` for in_range()
   - ✓ Use `dateFromTo` for BETWEEN
2. **Missing paramNames for dateFromTo**
   - ✓ Ensure both `from` and `to` are specified
3. **Date format mismatch**
   - ✓ Check `dateFormat` matches SQL expectations

Problem: Required Card Not Auto-Rendering

Symptoms: - Button marked as `required: true` but no card appears

Causes & Solutions:

1. **DragNDrop.renderRequiredCards() not called**
   - ✓ Check console for “Checking for required cards”
   - ✓ Verify `draw()` is called in DragNDrop
2. **Button not found**

- ✓ Ensure `paramName` exists in config
- ✓ Check getAllButtonConfigs() returns button

Problem: Preset Not Working

Symptoms: - Clicking preset does nothing - Preset creates wrong cards

Causes & Solutions:

1. **Invalid paramName in preset**
   - ✓ Verify each `paramName` exists in buttonGroups
   - Check console: “Button config not found for: X”
2. **Preset not linked to button**
   - ✓ Check `_presetConfig` is attached in LeftPane.js
3. **Duplicate prevention**
   - ✓ Cards already exist → preset won’t recreate them

Best Practices

1. Naming Conventions

Parameters: - Prefix with `P_` (e.g., `P_Marke`, `P_Year`) - Use descriptive names - Match report parameter names exactly

Queries: - Prefix with `qp_` (e.g., `qp_Marke`, `qp_ModelYear`) - Name matches parameter (minus prefix) - Keep names short but clear

DataSets: - Use exact query name (e.g., `qp_Marke`) - Case-sensitive matching

---

2. Query Design

Column Order:

```
SELECT
  UseValue,      -- Column 0 (for parameter)
  DisplayValue   -- Column 1 (for user)
FROM Table
ORDER BY DisplayValue
```

Performance: - Add WHERE clause to limit rows - Use indexes on filter columns - Consider views for complex queries

Size Limits: - < 1,000 rows: Excellent - 1,000 - 10,000 rows: Good - > 10,000 rows: Use `promptType: "text"`

---

3. Multi-Language

Always provide: - English (`en`) as fallback - German (`de`) if target users are German

Pattern:

```
{
  "label": "English Text",
  "labels": {
    "de": "Deutscher Text",
    "en": "English Text"
  }
}
```

Don't localize: - `paramName` (must match Cognos) - `queryName` (must match DataSet) - `id` (technical identifier)

---

4. Help Text

Good help text: - ✅ Explains what parameter does - ✅ Mentions multi-select vs single-select - ✅ Notes required fields - ✅ Clarifies technical details (in_range vs BETWEEN)

Example:

```
"helpText": "Multi-select: Choose one or more production years. Leave empty to include all y
```

---

5. Required Parameters

Use `required: true` **for:** - Mandatory filters (e.g., date range) - Security parameters (e.g., user region) - Performance-critical filters (prevents full table scan)

Combine with `maxValues: 1` **when:** - Single selection required - Prevents query ambiguity

---

6. Preset Design

Organize by use case: - "Basic Selection" → Common everyday filters - "Advanced Analysis" → Specialized combinations - "Quick Reports" → Preset date ranges + required params

Keep presets focused: - 3-5 parameters per preset - Related parameters only - Clear naming and descriptions

---

7. Testing Checklist

✅ All buttons appear in left pane
✅ Drag-and-drop works for all buttons
✅ Double-click works for all buttons
✅ Presets create correct cards

✓ Required cards auto-render
✓ Autocomplete shows correct values
✓ Use values (not display) sent to Cognos
✓ Date pickers accept input
✓ Remove button works (non-required)
✓ Required indicator changes to checkmark
✓ Multi-language displays correctly
✓ Paste support works (comma/newline)
✓ Parameters pass to report successfully

Appendix: Complete Minimal Configuration

```
{
  "BaseScriptPath": "/cognos4/samples/javascript/CustomPromptPage/",
  "selectFirstGroupExpanded": true,

  "presets": [
    {
      "id": "preset_basic",
      "label": "Basic Filters",
      "description": "Load common parameters",
      "parameters": [
        { "paramName": "P_Year" },
        { "paramName": "P_Brand" }
      ]
    }
  ],

  "buttonGroups": [
    {
      "groupLabel": "Filters",
      "defaultExpanded": true,
      "groupItems": [
        {
          "label": "Year",
          "paramName": "P_Year",
          "queryName": "qp_Years",
          "required": true,
          "maxValues": 1
        },
        {
          "label": "Brand",
          "paramName": "P_Brand",
          "queryName": "qp_Brands",
          "useColumn": 0,
          "displayColumn": 1
        },
        {
          "label": "Date Range",
          "paramName": "P_DateRange",
          "promptType": "dateRange",
          "dateFormat": "yyyy-MM-dd"
        }
      ]
```

```
    }
  ]
}
```

Support & Feedback

For questions or issues: 1. Check console logs for error messages 2. Verify DataSet configuration in Cognos 3. Review this documentation 4. Contact your Cognos administrator

Console Debugging: - Enable browser developer tools (F12) - Check Console tab for `[RightPane]`, `[LeftPane]`, `[DragNDrop]` messages - Look for ✖ error symbols

---

End of Documentation